

Golden Patents Manual

How to take one patent and the JSON the pipeline extracted from it, and turn that JSON into gold: every reaction, compound and pathway the patent describes, every field right, every value traceable to a line.

YOU NEED

the patent folder under `runs/`, the prompt files, Claude Code, and the findings workbook

PASSES

3 phases × 3 tracks. Each gate in three steps: validate the flags, you decide, apply the fixes. At the end, a semi-final verification, then a final verification by two subagents told to break the gold

YOU PRODUCE

gold `reactions.json`, `compounds.json`, `pathways.json`, plus a complete audit trail

YOUR ROLE

Claude reads, flags and validates its own flags. You decide every one. Nothing changes in the gold until you say so

Contents

1	What gold means and how the work is shaped	p. 2	7	Phase 3 · Semi-final and final verification (P9–P11)	p. 8
2	Before you start: prerequisites and folder layout	p. 3	8	Pathways track (P12–P15, proposed)	p. 9
3	The process map	p. 4	9	The gate: validate (P7), decide, fix (P8)	p. 10
4	Rules that hold on every pass	p. 4	10	The audit trail: files, workbook, templates	p. 11
5	Phase 1 · The count (P1–P4)	p. 5	11	Finishing: definition of done and hand-over	p. 12
6	Phase 2 · The content (P5, P6)	p. 7	A	Prompt index	p. 13
			B	Glossary	p. 13

The prompts themselves are in `prompts-for-gold/`, not in this manual. P1 to P15 here name the steps. Appendix A shows which file each step uses.

CHAPTER 1

What gold means and how the work is shaped

The pipeline reads a patent and writes three JSON files: reactions, compounds and pathways. They are a first draft written by a model. Gold is that draft after a second reader has proven it against the patent, line by line.

A record is gold when

- Everything the patent says about it is in the record.
- Nothing in the record is unsupported by the patent.
- Every empty field was checked, and the patent really is silent.
- Every generated label is right, and consistent across the file.

A file is gold when every record is, and it holds each reaction, compound or pathway the patent describes once. Deliberate exceptions are written down.

Two questions, in order

1. **Coverage.** Does the file hold everything the patent describes, and nothing more? Read the patent first, then compare.
2. **Content.** Is every field of every record right and complete? One record at a time, against the whole patent.

Coverage comes first. Checking fields in a file with missing records is wasted work.

Three phases

PHASE	CLAUDE	YOU
1 · Count	reads, lists, reconciles, flags	decide flags, order fixes
2 · Content	verifies each record, flags	decide flags, order fixes
3 · Decide	semi-final: re-derives every field, corrects. Final: two subagents try to fail the gold	accept, declare gold

Phases 1 and 2 flag and stop. Only Phase 3 corrects as it goes. That order keeps a bad correction out of the ground truth.

Every gate has three steps: Claude validates the flags (P7), you decide, Claude applies the fixes (P8).

Chapter 9.

Three tracks

Reactions, compounds, pathways. Each track has the same four passes: identify, reconcile, verify records, final check. In every phase, reactions run before compounds, and compounds before pathways, because each depends on the one before.

Where you fit

Claude reads and flags. You decide. Nothing changes in the gold until you say so.

Two caveats. No human has read the patent line by line unless you do. And the extraction and the audit are both Claude, so the independent read in Phase 1 is the only defence against a shared misreading. Keep it independent.

CHAPTER 2

Before you start

What you need on your machine

- ☐ **The repository** `github.com/yashmstack/patent-extraction`, cloned, with the pipeline already run for your patent. You check the output; you do not run the extraction.
- ☐ **Claude Code**, installed and signed in.
- ☐ **The prompts** in `prompts-for-gold/`. Read its README once.
- ☐ **The findings workbook** `GOLDEN-DATASET-FINDINGS.xlsx`, with a sheet for your patent (columns in Chapter 10).

What you need to know

- ☐ Enough organic chemistry to tell a reactant from a solvent, a range from a point value, and two names for one compound from two compounds.
- ☐ How to read the enriched markdown. Every line has a number, Chinese lines are followed by an indented `> EN:` translation, and drawings arrive as `[IMAGE_EXTRACT: ...]` blocks carrying SMILES read off the page.
- ☐ That the patent, not the pipeline and not the schema, is the standard. If the patent says it

and the file cannot hold it, that is a finding, never a reason to drop it.

The three inputs, and where they are

INPUT	PATH UNDER RUNS/<PATENT_ID>/
The patent, as downloaded	<code>input/pdf/</code>
Line-numbered translated markdown. The only source Claude reads.	<code>input/<PATENT_ID>-enriched-numbered.md</code>
Extracted reactions, the file under test	<code>output/reactions.json</code>
Extracted compounds, the file under test	<code>output/compounds.json</code>
Extracted pathways, the file under test	<code>output/pathways.json</code>

The PDF is for you. When a value looks wrong in the markdown itself, open the page in the PDF and look. The markdown came from a vision pass over scanned pages and can be wrong in ways Claude cannot see, because Claude reads the markdown.

Set up the folder

Open a terminal at the repository root and run:

```
cd runs/<PATENT_ID>
mkdir -p audit
cp output/reactions.json audit/reactions.json.ORIGINAL
cp output/compounds.json audit/compounds.json.ORIGINAL
cp output/pathways.json audit/pathways.json.ORIGINAL
claude
```

The `audit/` folder is where every list, log and note from this manual goes. The three `.ORIGINAL` copies are the extraction as delivered; nothing in this

process may edit them, and the finished README will quote counts against them.

The folder when you are done

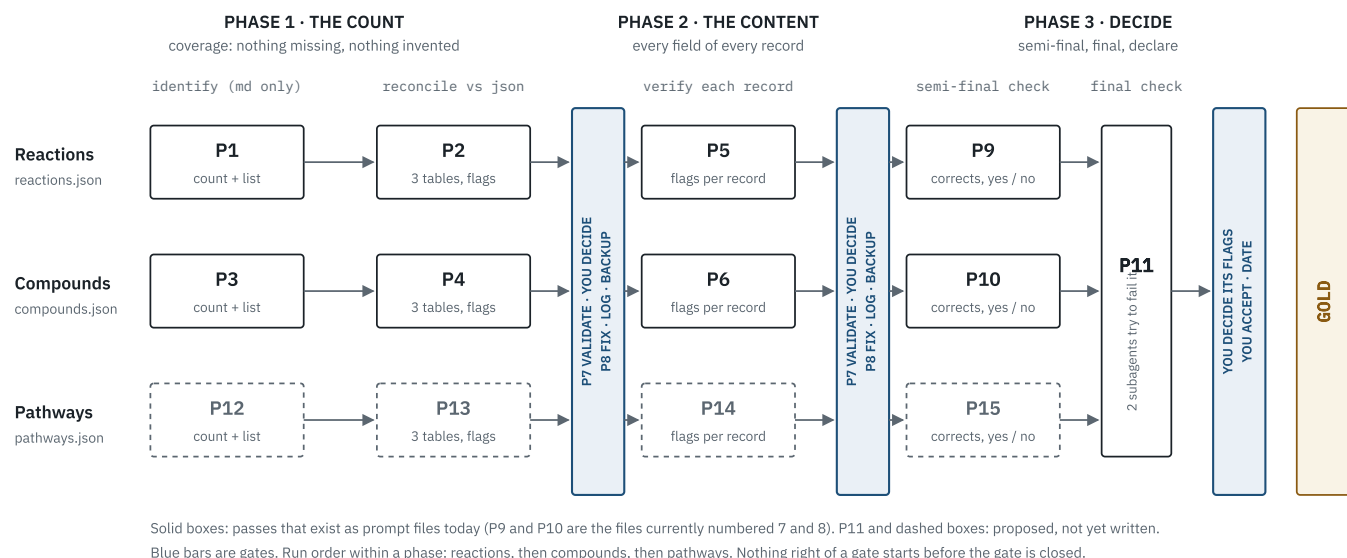
```
runs/<PATENT_ID>/
input/
  pdf/                                the patent
  <ID>-enriched-numbered.md          the source of every value
output/
  reactions.json  compounds.json  pathways.json    ← the GOLD
  reactions.schema.json          extended if fields were added
audit/
  reactions-read-from-patent.md    P1 independent list, written before the JSON was opened
  compounds-read-from-patent.md   P3 independent list
  pathways-read-from-patent.md    P12 independent list
```

AUDIT-LOG.md	per-record verification log, P5-P11
RUN-NOTES.md	what was odd about this patent, in prose
GOLDEN-DATASET-FINDINGS.xlsx	every flag, every decision
*.ORIGINAL, *.BACKUP-pre-<fix>.json	a copy before every fixing pass
README.md	what is here, what is not, caveats

When the patent is accepted, `input/`, `output/` and `goldenPatents/<PATENT_ID>/` and the root README
`audit/` are copied into table gets a row. Chapter 11.

CHAPTER 3

The process map



One sheet of the process. Each cell is one prompt pasted into Claude Code. Each blue bar is a gate: Claude validates the flags, you decide, Claude applies what you decided.

CHAPTER 4

Rules that hold on every pass

These are the rules the prompts enforce. They are here so you can tell when Claude is breaking one, and so you do not break one yourself while deciding flags.

- The identification pass never opens the extraction output.** Once a reader has seen the answer it finds the same things, and its count is worthless. In one session the same model produced counts of 48, 54 and 56 depending on what it had seen. If Claude admits it has seen the JSON, start a fresh session.
- Reconcile by identity, never by number.** Two files can hold 54 records each and disagree about half. DMF and N,N-dimethylformamide are one compound. Match on what reacts and what forms, not on labels or counts.
- Flag, do not fix,** in Phases 1 and 2. Fixing happens only after you have reviewed the flags. Phase 3 corrects as it goes, and logs every change.
- Never treat the pipeline's behaviour as correct.** It is the thing being measured. Look up what a field is defined to hold; never conclude a value is absent for a good reason because the pipeline left it out.
- A null is not evidence the patent is silent.** Every empty field is a question. Checking is the only way to tell an absent value from an uncaptured one.
- Prose is not capture.** A value sitting only in `notes` or a `*_text` field is invisible downstream. It is still a finding.
- Never withdraw a finding because the schema has no field.** Flag it. If it matters, the schema is extended. The gold must be more complete than the pipeline, not equally lossy.
- A range reaches the JSON as two numbers.** Min and max, with open bounds keeping their side. A single float holding one end of a range is a loss that looks like a value.
- A step told twice covers itself.** Where the claims and the summary both record one step, a value present on one is not missing from the other. Read twins together.
- List every item individually.** If there are 43 matches, the table has 43 rows. Ranges of rows hide exactly what you came to check.
- Log as you read.** Passes as well as failures, so the work can be checked later by someone who was

not there.

CHAPTER 5 • PHASE 1

The count: does the file hold everything the patent describes?

Four prompts, two per track. The odd-numbered prompt reads the patent alone and writes an independent list. The even-numbered prompt opens the JSON and reconciles the two by identity. You close the phase by deciding every flag.

P1 Identify every reaction in the patent

REACTIONS

BEFORE	Fresh Claude Code session in <code>runs/<PATENT_ID>/</code> . Claude must not have opened <code>output/</code> in this session. Ask it to confirm.
PASTE	<code>prompts-for-gold/1-reactions-identification.md</code> , with your patent id filled in.
CLAUDE DOES	Reads the markdown line by line, keeping a running audit. Decides how many genuine reactions, transformations and steps the patent describes and where each one is. Dedups on identity: the same reaction described in four sections is one entry; the same reaction performed twice is two.
YOU RECEIVE	A count, and a list with one line per reaction: what reacts, what forms, which section, which line numbers. Mechanical counts alongside (drawn arrows, examples, claims).
SAVE AS	<code>audit/reactions-read-from-patent.md</code> . Write the count at the top. Date it.

◆ Gate 1a · Re-verify the list before it becomes the yardstick

Open a second fresh session, or a context-free subagent, that has also not seen the JSON. Give it the markdown and the list and ask it to confirm the count, find anything the list missed, and challenge anything on it that is not a real transformation. Reconcile the two lists yourself. Only a list that has survived a second independent reader goes into P2.

P2**Reconcile the list against** `reactions.json`**REACTIONS**

BEFORE P1 finished and the count written down. Same or new session; from here the JSON may be opened.

PASTE `prompts-for-gold/2-reactions-reconciliation.md`.

CLAUDE DOES Takes one JSON record at a time, reads what it claims to be, reads the cited lines, then re-reads the whole markdown with that record in mind, and decides which transformation on the list it is. Matches by identity, never by count or label. Checks twins together before flagging.

YOU RECEIVE Three tables, every row an individual item. **Matches:** your transformation, its `reaction_id`, and whether the match is exact or by identity. **Missing:** in the patent but not in the file, with line and a reason it was probably missed. **Extra:** in the file but not in the patent, each with a verdict of false record or Claude's own miss. Then the three counts.

SAVE AS Append the tables to `audit/reactions-read-from-patent.md`. Every Missing, Extra and Wrong row becomes a workbook row.

◆ **Gate 1b · Validate, decide, fix**

Validate (P7): run the findings validator over the Missing, Extra and Wrong rows. It re-reads each cited line and marks the row confirmed, withdrawn with a reason, or needs-decision.

Decide: for each confirmed Missing row, is it a genuine transformation? Order it added. For each Extra row: false record or duplicate? Remove, or merge if it is a strict subset of its twin, or keep with a note if it is a real retelling that carries substance. For each Wrong row, order the correction. Write every decision in the workbook. **Fix (P8):** only then tell Claude to apply the decided changes, back up first, log each against its id, and re-run the three counts to prove the file now matches the list.

P3**Identify every compound the patent mentions****COMPOUNDS**

BEFORE Fresh session. Claude must not have opened `compounds.json`. The reactions work does not contaminate this pass, but a session that has seen `compounds.json` does.

PASTE `prompts-for-gold/3-compounds-identification.md`.

CLAUDE DOES Reads every line and writes down every compound with its line number: named substances, drawn structures, reagents, solvents, catalysts, and the class terms the claims recite ("an oxidant", "a base") even where no substance is named.

YOU RECEIVE A count and a list: compound, first line, every other line, the names the patent uses for it, the role it plays.

SAVE AS `audit/compounds-read-from-patent.md`. Count at the top.

◆ **Gate 1c · Re-verify the compound list**

Same as Gate 1a: a second reader that has not seen the JSON confirms the count and the identities. Pay particular attention to one substance under several names, and to isomers sharing a formula, which are two compounds, not one.

P4

Reconcile the list against `compounds.json`

COMPOUNDS

PASTE

`prompts-for-gold/4-compounds-reconciliation.md`,
after P3 is written down.

CLAUDE DOES

Record by record, matches on identity using `aliases` as well as `identifier`. Finds what the file has that the list has, what is in the file and not the patent, and what the patent names that the file does not hold. Flags duplicates: one substance written several times, once per naming form.

YOU RECEIVE

The same three tables and three counts. Duplicate groups listed with the records in each group and whether they genuinely differ in role or charge.

◆ Gate 1d · Validate, decide, fix

Same three steps. The two decision families that dominate here are the ones from the sketch on the desk: **duplicates** and **chemical structure**. Missing compounds are added, class terms included. Duplicates are merged losslessly only when nothing distinguishes them; a substance the patent charges in two roles or two sections stays split, and the reason is written into the record. Never merge on molecular formula: two isomers are two compounds. Log, then P8, then re-count.

Phase 1 is closed when

- Both independent lists exist, dated, each confirmed by a second reader.
- Every Missing, Extra and Wrong row has a decision in the workbook.
- The ordered fixes are applied, backups exist, and a re-count shows matched = list, missing = 0, extra = 0 apart from rows you closed by decision.
- Every `identifier` that `reactions.json` references resolves to a record in `compounds.json`.

CHAPTER 6 • PHASE 2

The content: is every field of every record right?

Phase 1 fixed the count. Now each record is checked in full. The unit of work is one record and one complete reading of the patent. If the file has 69 records, the patent is read 69 times, each time holding a different record in mind. This phase flags only.

The loop Claude runs, for every record

1. Open the file and read the schema actually present, so every field is judged by what it is defined to hold.
2. Take record *i*, starting at 0, in file order. Keep the index visible in every report so no record is skipped.
3. Understand the record: what it claims to be, what reacts, what forms, under what conditions.
4. Read the whole markdown, end to end, asking of every line whether it says anything about this record. Not only the cited lines: the charge is in the example, the range in the claims, the reason in the summary, the structure in a drawing, the meaning of R somewhere else.
5. Compare. Is every value correct? Is anything the patent says about this record missing? Is anything in the record unsupported? Is the record rich, carrying everything the patent gives and not the minimum?
6. Do steps 3 to 5 a second time. The second pass catches what the first missed.
7. Next record. When the last is done, return to 0 for a final sweep of the whole file.

What "read carefully" means here

Read normally first, for meaning: what the step is for, what a phrase refers back to, what a drawing shows. Then do the checkable things alongside. Count how many times the patent states a thing and check the file holds that many. Treat every quotation field as a claim and find the exact string. Line identical situations up side by side across records; most inconsistency is invisible one record at a time. Where a check can be mechanical, make it mechanical: counting, string matching and cross-record comparison are what a reader does worst and a short script does perfectly.

P5 Verify every record in `reactions.json`

REACTIONS

BEFORE	Phase 1 closed for reactions. Back up the file as <code>audit/reactions.BACKUP-pre-P5.json</code> .
PASTE	<code>prompts-for-gold/5-reactions-record-verification.md</code> .
CLAUDE DOES	The loop above. Pays special attention to empty fields, to ranges stored as single numbers, to preference tiers ("40–100 °C, preferably 70–90 °C" is two facts), to multi-stage one-pot runs whose stage conditions have nowhere to go, and to values that live only in prose.
YOU RECEIVE	The complete issue list, record by record, with index, <code>reaction_id</code> , field, line numbers, what the patent says, what the record has. The list of records that passed clean. Anything unresolved, with both readings.
SAVE AS	<code>audit/AUDIT-LOG.md</code> , one heading per record, passes included. Every issue becomes a workbook row on the Reactions half.

P6 Verify every record in `compounds.json`

COMPOUNDS

BEFORE	Phase 1 closed for compounds. Back up as <code>audit/compounds.BACKUP-pre-P6.json</code> .
PASTE	<code>prompts-for-gold/6-compounds-record-verification.md</code> .
CLAUDE DOES	The same loop, per compound. Checks role against what the compound actually does in each section, every quantity as a genuine pair from one real experiment, every analytics quotation verbatim, aliases complete, section anchors right, and that quantities of a solution are not read as quantities of the solute.
YOU RECEIVE	Issue list with index, <code>identifier</code> , field, lines; clean records; unresolved items.
SAVE AS	Append to <code>audit/AUDIT-LOG.md</code> . Workbook rows on the Compounds half.

◆ Gate 2 · Validate (P7), decide, fix (P8)

Run the findings validator over every P5 and P6 row first; a flag on a 1,100-line patent is sometimes a misread, and you should not spend a decision on it. Then work the confirmed and needs-decision rows top to bottom. For each row choose one decision from Chapter 9: fix as flagged, fix differently, keep and close by decision, add a field, or leave unresolved with both readings recorded. Where a field is needed that the schema lacks, decide now whether to extend the schema; the gold is allowed to hold more than the pipeline, never less. When every row has a decision, run P8: Claude applies the decided fixes in one pass, backs up first, logs each change against its finding id, and re-validates the JSON against the schema.

Phase 2 is closed when

- Every record has an entry in the audit log, pass or fail.
- Every issue is a workbook row with a decision.
- Ordered fixes are applied and logged, with a backup taken before.
- The files validate against their schemas, extended where you decided to extend.

CHAPTER 7 · PHASE 3

Semi-final and final verification: is it actually gold?

Two checks close the work. The **semi-final verification** is a prompt that assumes nothing: it walks every field of every record, asks where the answer comes from, and corrects as it goes. The **final verification** is two subagents with no context, each told to try to fail the gold. If neither can, it is gold.

Two kinds of field, checked two ways

A field the **patent must answer** (a mass, a temperature, a compound, a yield) is checked against the page. A field the **model generates** (reaction class, mechanism type, tags, confidence) is checked against the chemistry and against how the rest of the file treats the same situation. A generated label can be chemically defensible and still wrong, if two identical records carry different values.

P9

Semi-final verification of `reactions.json`

REACTIONS

FILE TODAY	7-reactions-final-gold-check.md
BEFORE	Every Phase 1 and Phase 2 flag validated, decided and applied. Back up as <code>audit/reactions.BACKUP-pre-final.json</code> .
CLAUDE DOES	Record by record from 0, reads the whole patent for each, then walks every field in order. Empty field: works out how it ought to be filled, verifies against the markdown, and either fills it or states that the patent is silent. Filled field: re-verifies, corrects if wrong, reasons why it was wrong, verifies the correction. Stops and asks when unsure rather than guessing.
YOU RECEIVE	Per record: passed clean or corrected, with index and lines. A plain statement: gold, or not gold and exactly what blocks it. Anything unsettled, with both readings.
SAVE AS	Append to <code>audit/AUDIT-LOG.md</code> . Every correction is a workbook row marked as made in the semi-final check.

P10

Semi-final verification of `compounds.json`

COMPOUNDS

FILE	TODAY	8-compounds-final-gold-check.md
BEFORE		Back up as audit/compounds.BACKUP-pre-final.json.
CLAUDE	DOES	The same re-derivation per compound: identity, aliases, role, section, every quantity, every analytics quotation, every derived field, every deliberate split, and the reason for every null.
YOU	RECEIVE	Per record verdict, the gold statement, unsettled items.

P11

Final verification: two subagents try to fail the gold

ALL TRACKS

FILE		proposed 11-all-final-verification.md ; today this is done by hand with fresh sessions
WHY		A fixing pass introduces defects of its own. On one finished patent four of the last eight defects were introduced by the annotation, not the extraction, and only a reader with no memory of the fixes caught them.
CLAUDE	DOES	Launches two subagents with no context. Each gets only the markdown and the finished files, and one instruction: try to prove this is not gold. Find any value that is wrong, missing or unsupported, with line numbers. They do not see the workbook, the audit log or each other. Their findings are merged into one triage list.
YOU	RECEIVE	The triage list, each item with the record, the line, the claim and which subagent raised it.

◆ Gate 3 · Decide the final-verification findings, accept the corrections, declare

Treat the triage list like any other flag list: validate, decide, fix. Then read every correction the semi-final check made against its line numbers. If either subagent found a real failure, the fix goes in and both subagents run again. When neither can fail it, write the acceptance sentence with your name and the date.

Phase 3 is closed when

- The semi-final checks P9 and P10 each end with the statement "gold", or every unsettled item has your written ruling.
- The final verification has run, both subagents failed to break the gold, and every item they raised is decided and logged with *introduced by* filled in.
- You have read the finished files and the findings yourself, and accepted them, with the date.

Declaring gold is your act, not Claude's. The sentence in the README is "the annotator read and accepted this on <date>". Write what you did and did not do: reading the findings and directing every judgement call is one thing, reading the patent line by line yourself is another, and the README says which.

CHAPTER 8 • PROPOSED

The pathways track

A pathway is a chain of reaction records from a key starting material through intermediates to a product, scoped to a section or a route. `pathways.json` is built on top of the other two files, so it is checked last and its prompts are not yet written. This chapter fixes the shape of that work and proposes the file names, so the prompts can be written to match.

What is different about pathways

A reaction record is judged against the page. A pathway record is judged against the page **and** against `reactions.json`: every step it names must resolve to a reaction record, the product of step n must be a reactant of step $n+1$, and the route must be one the patent actually draws or describes, not one assembled from steps that happen to connect. The independent read therefore lists routes as the patent tells them, scheme by scheme and section by section.

P12

Identify every pathway the patent describes

PATHWAYS

FILE	12-pathways-identification.md (proposed)
RULE	Fresh session; <code>pathways.json</code> not opened. <code>reactions.json</code> may be consulted, because a pathway is defined in terms of it.
OUTPUT	One row per route: section, starting material, ordered intermediates, product, the lines and drawn schemes that tell it. Count at the top. Saved as <code>audit/pathways-read-from-patent.md</code> .

P13

Reconcile the list against `pathways.json`

PATHWAYS

FILE	13-pathways-reconciliation.md (proposed)
OUTPUT	Three tables: matched routes, routes the patent tells that the file lacks, routes in the file the patent does not tell (typically two schemes merged, or a route stitched across sections). Flag only.
GATE	Validate (P7), decide, fix (P8), re-count.

P14

Verify every pathway record

PATHWAYS

FILE	14-pathways-record-verification.md (proposed)
CHECKS	Every step resolves to a <code>reaction_id</code> ; products chain into reactants; identifiers resolve to compound records; scope and section are right; the drawn route and the recorded route agree in order and length; nothing is inferred that the patent does not draw or say. Flag only.

P15

Final gold check on `pathways.json`

PATHWAYS

FILE	15-pathways-final-gold-check.md (proposed)
RULE	Re-derive every field, correct as you go, end with yes or no. Then the final verification (P11) covers this file too. Same gate as Chapter 7.

Naming follows the existing convention: number, track, pass. Numbers 12 to 15 leave room for the validator, fix and final-verification prompts at 7, 8 and 11 that the desk sketch calls for. Appendix A has the full proposed list, including the zero-padding that keeps 10 sorting after 9.

CHAPTER 9

The gate: validate, decide, fix

Every flagging pass ends with a list and stops. What happens next is the same at every gate: Claude validates its own flags, you decide what survives, Claude applies exactly what you decided. This chapter is that procedure, the five decisions you can make, and the mistakes made before.

P7

Findings validator: are the flags actually right?

ALL TRACKS

FILE

INPUT

CLAUDE DOES

YOU RECEIVE

proposed 07-all-findings-validation.md

The open workbook rows from the pass just finished, the markdown, the file under test.

For every row: re-reads the cited lines and the twin record, re-checks the arithmetic, and answers one question, is this finding true? Marks each row **confirmed**, **withdrawn** with the reason, or **needs-decision** where the page supports two readings or the answer is a convention rather than a fact. Groups rows that are one decision (the same lost range on seven records).

The workbook with the validator's verdict on every row and the groups marked. Nothing has been changed in the JSON.

reopen any you disagree with.

5. **Run P8** when every row has a decision.
6. **Read the diff** between backup and result. A fix that touched more than the flag asked for is itself a flag.

The five decisions

DECISION	MEANING	STATUS AFTER
FIX	The flag is right. Apply as described.	decided → fixed → verified
FIX-OTHER	Real problem, wrong proposed correction. Write the right one.	decided → fixed → verified
KEEP	The record is right as it stands. Say why; if the reason is a convention, write it into the record's notes so the next reader does not flag it again.	closed by decision
ADD FIELD	The patent states something the schema cannot hold. Extend the schema, describe the field, then FIX.	decided → schema → fixed
UNRESOLVED	Two readings, no way to choose from the page. Record both in notes. Never average them.	open, both readings

A flag is never deleted. A wrong flag is closed as **withdrawn** with the reason, by the validator or by you, because a withdrawn flag teaches the next reader as much as a fixed one.

Your procedure, per gate

1. **Read the whole validated list once** before deciding anything. Decide groups, not rows, where the validator grouped them.
2. **Open the page** for every confirmed and needs-decision row. Read the cited line in the markdown, and when the markdown is the thing in doubt, the PDF. Never decide from the flag's own summary.
3. **Check the twin.** A record with a sibling from another section is read with it before anything is called missing.
4. **Decide**, one of the five below, and write decision and reason in the row. Skim the withdrawn rows;

P8

Apply the decided fixes

ALL TRACKS

FILE	proposed 08-all-apply-decided-fixes.md
INPUT	The workbook rows with status <i>decided</i> , and nothing else. A row without a decision is not touched.
CLAUDE DOES	Copies the file to audit/<file>.BACKUP-pre-<gate>.json. Applies each decided row exactly as written, and only that. Writes a note on the record where the decision was KEEP or ADD FIELD. Re-validates against the schema, re-runs the mechanical counts and the identifier join, and logs every change against its finding id.
YOU RECEIVE	The change log, one line per finding id, and the diff summary: records touched, fields touched, counts before and after.

Decisions that come up on every patent

SITUATION	USUAL RULING
One step told in claims, summary and description	Keep a record per section when each carries substance the others lack. Merge only strict subsets.
One substance, several names	One record; other names go to aliases. Original-language names are kept as aliases, never dropped.
One substance, two roles or two charges in different sections	Keep split. Merging destroys a fact.
Two compounds, one molecular formula	Two records. Formula is never an identity key.
Drawn structure disagrees with the name	Check the PDF drawing atom by atom. The drawing usually wins over a machine-rendered name; record which you followed and why.
Range stored as one number	FIX to min and max; point field null. Open bound keeps its side, other end null.
Value only in prose or notes	FIX into a structured field, or ADD FIELD.
Patent's own arithmetic or printing is wrong	KEEP as printed, contradiction written into notes. Never silently correct the patent.
Machine translation names the wrong compound	Follow the original-language text; keep the wrong rendering as an alias with a note.
Field is general knowledge, not in the patent	Allowed, but the README must say the field is not a reading of the patent.

Mistakes already made once

- Letting the identification reader see the JSON. The count moved by eight.
- Merging on molecular formula and fusing two isomers into one record.
- Reading "below 10%" as a yield of 10.
- A fixing pass marking exact values approximate, and assigning an analytical method by section adjacency. Caught only by the final verification.
- Taking a wrong OCR of the product name at face value. It named a different registered herbicide.

CHAPTER 10

The audit trail: files, workbook and templates

Gold without a trail is an opinion. Every list, flag, decision and change lives in `audit/`, and a reader who was not there must be able to reconstruct why every record is the way it is.

The files, and who writes them

FILE	WRITTEN IN	HOLDS
<code>reactions-read-from-patent.md</code>	P1, P2	Independent list with count; then the three reconciliation tables
<code>compounds-read-from-patent.md</code>	P3, P4	Same, for compounds
<code>pathways-read-from-patent.md</code>	P12, P13	Same, for pathways
<code>AUDIT-LOG.md</code>	P5, P6, P9, P10, P11, P14, P15	One heading per record per pass: what was checked, what passed, what was flagged or corrected, with lines
<code>RUN-NOTES.md</code>	you, throughout	Prose. What is odd about this patent, what the vision pass got wrong, what conventions you set and why
<code>GOLDEN-DATASET-FINDINGS.xlsx</code>	every pass; P7 adds verdicts, P8 adds status	Every flag and its decision. One sheet per patent; reactions half, compounds half, pathways half
<code>*.ORIGINAL</code> , <code>*.BACKUP-pre-*.json</code>	P8 and the final check, before every fix	The file as it was. Never edited
<code>README.md</code>	you, at the end	What is here, what is not, counts, caveats, acceptance date

Workbook columns

One row per finding. Ids are prefixed by track and never reused: `F-001` reactions, `C-001` compounds, `P-001` pathways.

COLUMN	CONTENT
<code>id</code>	F-/C-/P- number
<code>prompt</code>	P1 ... P15, or GATE for findings you raised yourself
<code>record</code>	index and <code>reaction_id</code> or <code>identifier</code>
<code>field</code>	the JSON path, e.g. <code>conditions.time_h</code>
<code>line(s)</code>	markdown line numbers
<code>patent says</code>	quoted, in the patent's language plus English
<code>record has</code>	the current value, verbatim
<code>should be</code>	the proposed value
<code>class</code>	MISSING · WRONG · EXTRA · DUPLICATE · RANGE_LOST · NO_FIELD · PROSE_ONLY · INCONSISTENT · UNRESOLVED
<code>validator</code>	confirmed · withdrawn · needs-decision, with the P7 reason
<code>decision</code>	FIX · FIX-OTHER · KEEP · ADD FIELD · UNRESOLVED · WITHDRAWN, with reason
<code>status</code>	open · decided · fixed · verified · closed by decision
<code>introduced by</code>	extraction, or annotation (a defect a fixing pass created, usually found by the final verification)
<code>who / date</code>	your initials and the date of the decision

Write every row so a beginner with no context can understand what is wrong: what the patent says, what the record has, what it should be, and the line.

Template: independent list header

```
# <PATENT_ID> · P1 independent reaction list
Source: input/<ID>-enriched-numbered.md, N
lines, all read
Extraction output NOT opened. Disclosure:
<anything seen>
Date: YYYY-MM-DD   Reader: Claude session <n>
Re-verified by: second reader, YYYY-MM-DD,
agrees / differs (see below)

COUNT: <n> reactions   (<a> described, <b>
performed)

### Mechanical counts
### Compound key
### Reactions, one per line, with section and
lines
```

Template: audit log entry

```
### [P5] record 33 · Example 10 Step 1 · lines
537-541
Read: full md, pass 1 and pass 2.
Checked: compounds (5), conditions, workup,
yield, generated labels.
PASS: reactants, product, yield 76.59 (L541).
FLAG F-041 conditions.stages: two-stage run,
stage-1
      80-85 °C / 8 h absent (L537). class
NO_FIELD.
FLAG F-042 workup.ph_target: pH 8-9 then 2-3,
record
      holds 2.0 only (L537). class RANGE_LOST.
```

Template: README skeleton

```
# <PATENT_ID> · what is here and what is not
Both files are gold / reactions gold,
compounds pending.
Accepted by <name> on <date>. Read the
findings and
directed every judgement call; did / did not
read the
patent line by line independently.
      reactions.json   N records   (extraction
delivered M)
      compounds.json   N records   covering K
distinct compounds
### What the gold holds           ### Coverage,
checked mechanically
### Fields added and why         ### Empty fields
are empty on purpose
### Deliberate splits and merges
### Caveats (no human line read; extraction
and audit both Claude)
### Files
```

CHAPTER 11

Finishing: definition of done and hand-over

Definition of done, per patent

- ☐ Phase 1, 2 and 3 closed for reactions and for compounds, per the close-out boxes in Chapters 5 to 7. Pathways likewise once P12 to P15 exist.
- ☐ Every workbook row has a decision and a status other than open, or is marked UNRESOLVED with both readings in the record's notes.
- ☐ Both JSON files validate against their schemas with zero errors. If fields were added, the schema file is committed alongside and each new field carries a description saying what it holds and why the original could not.
- ☐ Every identifier referenced by `reactions.json` resolves to a record in `compounds.json`. Every reaction referenced by `pathways.json` resolves to a record in `reactions.json`.
- ☐ The final verification (P11) has run and neither subagent could fail the gold.
- ☐ `audit/README.md` written from the skeleton, with counts, additions, merges, splits, caveats and the acceptance sentence.
- ☐ `RUN-NOTES.md` records anything a future reader would otherwise rediscover the hard way.

Hand-over

1. Copy `input/pdf/<ID>.pdf`, the enriched numbered markdown, `output/reactions.json`, `output/compounds.json`, `output/pathways.json` and the whole `audit/` folder into `goldenPatents/<PATENT_ID>/` under `input/`, `output/`, `audit/`.
2. Add the patent's row to the table in the root `goldenPatents/README.md`: id, record counts, state. Add its paragraph below, in the form the

existing entries use: what the patent is, what the gold holds, what was changed and why, what is deliberately not here.

3. Commit. The commit message names the patent and the acceptance date.

Working across sessions

Context runs out. Before a session ends, ask Claude to write where it is (phase, prompt, record index, open flags) to `RUN-NOTES.md`. The next session starts by reading that, the independent list, and the audit log for the current pass. An identification pass that is interrupted is restarted from the top in a fresh session; a verification pass resumes at the recorded index.

Working in parallel

The reactions and compounds tracks are independent through Phase 2 and can run in two sessions at once. They meet at Gate 2, because a compound added or merged changes what a reaction record's identifiers resolve to. Do not run Phase 3 on either file until Phase 2 is closed on both.

When the markdown is wrong

Claude reads the markdown, not the PDF. When a flag makes you suspect the markdown itself, look at the PDF page. If the vision pass misread something, the fix belongs in `input/` (the vision JSON and the regenerated markdown), it is written up in `RUN-NOTES.md`, and any records that depended on it go back through P5 or P6. The original-language text is authoritative; the machine English is a convenience.

The one sentence to remember

Claude reads and flags. You decide and log. Nothing enters the gold that you did not decide, and nothing you decided is without a line number.

APPENDIX A

Prompt index

Numbers in this manual are roles, and the roles are stable even while the files are being revised. The *file today* column is the folder as it is on 2026-09-03. The *proposed name* column is one suggestion for the revised set: zero-padded so 10 sorts after 9, track second so a sort by name groups a track, and `all` as the track for passes that run over every file.

ROLE	WHAT IT DOES	PHASE	TRACK	FILE TODAY	PROPOSED NAME	OPENS JSON UNDER TEST	FLAGS / FIXES
P1	read the patent, count the reactions	1 Count	reactions	1-reactions-identification.md	01-reactions-identification.md	never	lists
P2	reconcile the list against reactions.json	1 Count	reactions	2-reactions-reconciliation.md	02-reactions-reconciliation.md	yes	flags
P3	read the patent, list the compounds	1 Count	compounds	3-compounds-identification.md	03-compounds-identification.md	never	lists
P4	reconcile the list against compounds.json	1 Count	compounds	4-compounds-reconciliation.md	04-compounds-reconciliation.md	yes	flags
P5	verify every field of every reaction record	2 Content	reactions	5-reactions-record-verification.md	05-reactions-record-verification.md	yes	flags
P6	verify every field of every compound record	2 Content	compounds	6-compounds-record-verification.md	06-compounds-record-verification.md	yes	flags
P7	findings validator: is each flag actually right?	every gate	all	not yet a file	07-all-findings-validation.md	yes	verdicts on flags
P8	apply the fixes you decided, backup, log	every gate	all	not yet a file	08-all-apply-decided-fixes.md	yes	fixes , decided only
P9	semi-final verification, re-derive every field	3 Decide	reactions	7-reactions-final-gold-check.md	09-reactions-semifinal-verification.md	yes	fixes , logged
P10	semi-final verification, re-derive every field	3 Decide	compounds	8-compounds-final-gold-check.md	10-compounds-semifinal-verification.md	yes	fixes , logged
P11	final verification: two subagents try to fail the gold	3 Decide	all	not yet a file	11-all-final-verification.md	yes	flags
P12	read the patent, list the pathways	1 Count	pathways	—	12-pathways-identification.md	never	lists
P13	reconcile against pathways.json	1 Count	pathways	—	13-pathways-reconciliation.md	yes	flags
P14	verify every pathway record	2 Content	pathways	—	14-pathways-record-verification.md	yes	flags
P15	semi-final verification of	3 Decide	pathways	—	15-pathways-final-gold-	yes	fixes , logged

Every prompt ends by asking you whether anything is unclear before it starts, and tells Claude to stop and ask rather than guess when stuck. Answer those questions in the session and copy anything that amounts to a convention into `RUN-NOTES.md`.

APPENDIX B

Glossary

gold	A file every record of which meets the five conditions in Chapter 1, accepted by a named person on a date.
extraction	The pipeline's output, the JSON under test. Never a standard.
enriched numbered markdown	The patent as text, one number per line, original language followed by an indented English line, drawings as <code>IMAGE_EXTRACT</code> blocks. The only source Claude reads.
record	One object in a JSON array: one reaction, one compound, one pathway.
identifier	The compound name a record is keyed on. The join between <code>reactions.json</code> and <code>compounds.json</code> , matched on identifier or alias.
alias	Another name for the same substance on the same record: the original-language string, a synonym, a SMILES, an InChIKey.
described vs performed	A reaction told in prose (claims, summary, description) versus one actually run in an example with charges and a yield. Described repeats are one entry; performed repeats are one each.
twin	Two records of one step from different sections. Read together; they cover each other.

split	One substance deliberately held as two records because it plays two roles or is charged two ways. Disambiguate on role and section.
class term	A word the claims recite in place of a substance: "an oxidant", "a base". Recorded as a compound with no structure.
range	Two numbers. Stored as min and max; the point-valued field stays null. An open bound has one side null.
generated field	A label the model assigns (reaction class, mechanism, tags). Checked against chemistry and against sibling records, not against the page.
flag	A finding raised by a pass. Becomes a workbook row. Never deleted; withdrawn with a reason if wrong.
gate	The point at the end of a phase where you decide every flag before anything changes.
closed by decision	A flag you ruled KEEP, with the reason written into the record and the workbook.
findings validator	The pass (P7) that re-checks every flag against the page before you see it, and marks it confirmed, withdrawn or needs-decision.
semi-final verification	The prompt (P9, P10) that re-derives every field of every record and corrects as it goes.
final verification	Two subagents with no context (P11), told to try to fail the gold. If neither can, it is gold. The defence against defects the fixing passes introduced.
context-free reader	A fresh Claude session or subagent that has seen none of the earlier work, used to re-verify a list or a finished file.